moonmart.ai — Complete Application Workflow

> *Platform: moonmart.ai (formerly SaasWorld) Stack: Nuxt 3, SQLite (better-sqlite3), Stripe, Claude AI*

Table of Contents

1. User Roles & Access Levels
2. Common Features (All Users)
3. Buyer Workflow
4. Vendor Workflow
5. Admin Workflow
6. Authentication Flows
7. AI Features
8. Billing & Subscription Flow
9. Trust & Verification System
10. Developer / API Access

---

1. User Roles & Access Levels

Role	Description	Primary Dashboard
`buyer`	Software purchasing professionals evaluating tools	`/dashboard/overview`
`vendor`	Software companies listing and managing their products	`/dashboard/products`
`admin`	Platform moderators and administrators	`/dashboard/pending-apps`

Role is stored in `users.role` (SQLite). Admin grants elevated roles via `PUT /api/admin/users/[id]/role`.

---

2. Common Features (All Users)

These features are available to all visitors (authenticated or not), unless noted.

2.1 Marketplace Browsing

Route: `/marketplace`
API: `GET /api/apps`

Steps:

1. Land on `/marketplace` — full app catalog loads with server-side rendering
2. Apply filters: category, pricing type (free/paid/freemium), rating, tags
3. Sort by: trending, featured, newest, top-rated
4. Paginate through results (server-side pagination)
5. Click any app card → navigates to `/marketplace/app/[id]`

2.2 App Detail Page

Route: `/marketplace/app/[id]` or `/app/[id]`
API: `GET /api/apps/[id]`

View event: `POST /api/apps/[id]/view` (auto-fires on load)

Sections shown:

- Header: name, logo, tagline, rating, review count, pricing, verification badges
- Features: key feature list
- Screenshots gallery
- Pricing tiers
- Integrations
- Reviews (with authenticity scores)
- Review synthesis (AI-generated consensus)
- Similar apps
- Vendor info

2.3 Category Browsing

Routes: `/marketplace/category` → `/marketplace/category/[id]`
API: `GET /api/apps?category=[id]`

2.4 Global Search

Component: `GlobalSearch.vue` (client-only overlay)
Steps:

1. Press `/` or click search icon to open global search modal
2. Type app name, category, tag, or vendor
3. Results populate in real-time
4. Click result → navigates to app or category page

2.5 Alternatives & Comparisons

Routes: `/alternatives`, `/alternatives/[slug]`
Purpose: Side-by-side comparisons of competing tools

2.6 Reviews (Read)

Route: `/reviews`, embedded in app detail
API: `GET /api/apps/[id]/reviews`
Each review shows:

- Star rating (1–5)
- Pros / Cons
- Verified badge (if purchase confirmed)
- Authenticity score
- Helpful vote count

2.7 Q&A Forum

Routes: `/qa`, `/qa/[slug]`, `/qa/ask`
APIs:

- `GET /api/qa/questions` — browse all questions
- `GET /api/qa/questions/[slug]` — view thread
- `POST /api/qa/vote` — upvote/downvote (auth required)

Steps:

1. Browse or search questions on `/qa`
2. Click question → view full thread with answers
3. Authenticated users can answer, upvote, and mark accepted answer

2.8 Blog & News

Routes: `/blog`, `/blog/[slug]`, `/news`, `/news/[slug]`
APIs: `GET /api/blog/[slug]`, `GET /api/news/[slug]`
News supports reactions (upvote/downvote) via `POST /api/news/[id]/react`.

2.9 Buying Rooms (Collaborative Evaluation)

Route: `/buying-room`, `/buying-room/[slug]`
Steps (view):

1. Browse public buying rooms at `/buying-room`
2. Click room → see apps under evaluation, vote scores, comments
3. Auth required to participate (see Buyer Workflow §3.7)

2.10 RFPs (Request for Proposals)

Routes: `/rfp`, `/rfp/[slug]`
Steps (view):

1. Browse public RFPs at `/rfp`
2. Click RFP → see requirements, budget, deadline, responses
3. Auth required to submit or respond (see §3.8 and §4.9)

2.11 Stack Directory

Route: `/stack`
Public showcase of popular software stacks. Powered by `stack_overlaps` co-occurrence data.

2.12 Guides & Changelog & Roadmap

- `/guides` / `/guides/[slug]` — How-to guides, powered by `GET /api/guides`
- `/changelog` — Product updates, `GET /api/changelog`
- `/roadmap` — Upcoming features with voting, `GET /api/roadmap` + `POST /api/roadmap/vote`

2.13 GDPR & Consent

Component: `GdprConsentBanner.vue`
API: `POST /api/consent`
On first visit: consent banner appears. User accepts/declines → preference stored server-side.

2.14 Localization

Plugin: `global-market.ts`
Composable: `useGlobalMarket()`

- Currency conversion applied to all pricing displays
- Language switcher via `LanguageRegionSwitcher.vue`
- Supported locales include `en`, `fr`, `hi`, `ko`
- hreflang tags injected via `useHreflang()`

---

3. Buyer Workflow

3.1 Registration & Onboarding

Step 1 — Sign Up

- Route: `/signup`
- API: `POST /api/auth/register`
- Inputs: first name, last name, email, password, company name, job title
- Role assigned: `buyer` by default

Step 2 — Email Verification

- Verification link sent to inbox
- API: `GET /api/auth/verify-email?token=…`
- In dev mode: auto-verified (no SMTP needed)
- Resend: `POST /api/auth/resend-verification`

Step 3 — Login

- Route: `/login`
- API: `POST /api/auth/login`
- Session cookie set (14-day TTL)
- Account lockout after 10 failed attempts in 30 min (15-min lockout)

Step 4 — OAuth Alternative

- GitHub: `GET /api/auth/oauth/github` → callback
- Google: `GET /api/auth/oauth/google` → callback
- Account created or linked on first OAuth login

Step 5 — Onboarding Form

- Route: `/onboarding/submit`
- API: `POST /api/onboarding/submit`
- Captures: company size, use case, tool categories of interest

---

3.2 Dashboard Overview

Route: `/dashboard/overview`
Composable: `useBuyerData()`

On load, buyer dashboard shows:

- Saved apps count
- Active enquiries
- Stack spend summary
- Latest AI recommendations
- Digest items
- Recent activity

3.3 Saving Apps & Wishlist

Route: `/dashboard/stack` (saved section)
APIs:

- `POST /api/user/favorites` — add to favorites
- `GET /api/user/favorites` — fetch saved list
- `DELETE /api/user/favorites` — remove
- `GET /api/buyer/saved-apps` — richer saved app view
- `PATCH /api/buyer/saved-apps/[appId]` — update notes or status

Steps:

1. On any app card or detail page, click "Save" / bookmark icon
2. App stored in `user_favorites` table
3. View all saved apps at `/dashboard` → Saved section
4. Add personal notes, change status (evaluating, rejected, shortlisted)

3.4 Writing a Review

Route: App detail page → Reviews tab
API: `POST /api/reviews`

Steps:

1. Navigate to app detail page (`/marketplace/app/[id]`)
2. Click "Write a Review"
3. Fill in:
    - Star rating (1–5)
    - Title and body
    - Pros and cons
    - Use case description
    - Outcome metric (optional)
    - Company size and role
4. Submit → stored in `reviews` table with status `pending`
5. Trust engine scores authenticity (`computeReviewAuthenticity()`)
6. If buyer email matches vendor customer list → `purchase_verified = true`
7. Admin approves → review goes live

Marking reviews helpful:

- `POST /api/reviews/[id]/helpful`
- Increments `helpful_votes`

Flagging inappropriate reviews:

- `POST /api/reviews/[id]/flag`
- Increments `flag_count` ; admin sees flagged reviews in moderation queue

---

3.5 Building a Software Stack

Route: `/dashboard/stack`
APIs:

- `GET /api/stack` — load current stack
- `POST /api/stack` — add app
- `DELETE /api/stack/[id]` — remove app
- `POST /api/stack/[id]/reminder` — set renewal reminder

- `GET /api/stack/benchmark` — compare spend to similar companies
- `GET /api/stack/similar` — see what similar buyers use

Steps:

1. Go to `/dashboard/stack`
2. Click "Add Tool" — search and select an app
3. Enter: price (USD), billing period, seats, renewal date, notes
4. Stack saved in `user_stacks` table
5. Stack benchmarking: `GET /api/stack/benchmark` compares your spend vs. similar profiles
6. Set renewal reminder → scheduled email fires before renewal date (via `renewal-reminders.ts` task)
7. Stack overlap alerts: `buyer_stacks.overlap_alerts` flags duplicate-purpose tools

3.6 Sending an Enquiry to a Vendor

Route: App detail page → "Contact Vendor" button
APIs:

- `POST /api/enquiries` — create enquiry
- `GET /api/enquiries/[id]/messages` — view thread
- `POST /api/enquiries/[id]/messages` — send reply

Steps:

1. On app detail page, click "Contact Vendor" or "Request Demo"
2. Fill enquiry form: subject, message, optional budget
3. Enquiry created in `enquiries` table; email notification sent to vendor
4. Buyer tracks thread at `/dashboard/enquiries`
5. Vendor replies → both sides see the conversation thread
6. Status progresses: `open` → `in_progress` → `closed`

3.7 Creating a Buying Room (Team Evaluation)

Route: `/buying-room`, `/dashboard`
APIs:

- `POST /api/buying-rooms` — create room
- `POST /api/buying-rooms/[slug]/apps` — add app to room
- `POST /api/buying-rooms/[slug]/apps/[roomAppId]/vote` — vote on app
- `POST /api/buying-rooms/[slug]/comments` — add comment
- `POST /api/buying-rooms/[slug]/invite` — invite team member

Steps:

1. Click "Create Buying Room" from dashboard or `/buying-room`
2. Name the room, add description
3. Invite teammates by email → stored in `buying_room_members`
4. Add apps to evaluate → stored in `buying_room_apps`
5. Team members vote (👍/👎) on each app
6. Leave comments per app
7. Room shows aggregate vote scores for final decision
8. Room status: `active` → `decided` → `archived`

3.8 Submitting an RFP

Route: `/rfp` → "Post RFP" button
API: `POST /api/rfp` (rate limited: 10/hour)

Steps:

1. Navigate to `/rfp`, click "Post RFP"
2. Fill in:
    - Title and category
    - Budget range (min/max, currency)
    - Seat count
    - Requirements (markdown body)
    - Deadline
3. RFP published to `/rfp/[slug]`
4. Vendors see it in their dashboard and can respond
5. Buyer reviews vendor responses at `/rfp/[slug]`

3.9 AI Copilot (Shopping Assistant)

Route: `/dashboard/copilot`
API: `POST /api/ai/copilot` (rate limited: 60/hour)
Composable: `useAICopilot()`

Steps:

1. Open Copilot at `/dashboard/copilot`
2. Describe your requirement in natural language: "I need a CRM for a 50-person B2B team under $50/seat"
3. AI sends messages to Claude, which returns matched apps with reasoning
4. Session stored in `ai_match_sessions` table (for history and lead scoring)
5. Click any recommended app to view detail or save
6. Generate buying brief: `POST /api/ai/briefing` — AI drafts an executive brief

---

3.10 AI Negotiation Tips

Route: App detail page → "Negotiate Price" or `/dashboard`
API: `POST /api/ai/negotiation`

Steps:

1. On app detail, click "Get Negotiation Tips"
2. AI analyzes vendor pricing, market rates, review data
3. Returns negotiation script: opening offer, leverage points, walk-away price
4. Brief stored in `negotiation_briefs` table

3.11 Deals & Discovery

Route: `/dashboard/deals`, `/dashboard/discovery`
APIs:

- `GET /api/buyer/stack` — current stack context

- `GET /api/apps/[id]/similar` — similar apps discovery

Steps:

1. `/dashboard/deals` — shows curated deals and discounts relevant to your stack
2. `/dashboard/discovery` — new apps matching your profile and past behavior
3. AI recommendations at `/dashboard/recommendations` — personalized picks

3.12 Contracts Management

Route: `/dashboard` (contracts section)
APIs:

- `GET /api/contracts` — list contracts
- `POST /api/contracts` — add contract
- `PATCH /api/contracts/[id]` — update
- `DELETE /api/contracts/[id]` — remove

Steps:

1. Add a contract: vendor name, app, price, seats, start/end date, auto-renews flag
2. Stored in `contracts` table
3. Renewal reminders fire automatically before `end_date`
4. Status: `active` → `expiring_soon` → `expired`

3.13 Support Tickets

Route: `/dashboard/support`
API: `POST /api/support/tickets` (rate limited: 5/hour)

Steps:

1. Click "New Ticket" at `/dashboard/support`
2. Select category, write description
3. Ticket created in `support_tickets` table
4. Track replies in ticket thread
5. Close ticket when resolved

---

4. Vendor Workflow

4.1 Registration & Profile Setup

Step 1 — Register as Vendor

- Route: `/signup` → select "I'm a Vendor"
- API: `POST /api/auth/register` with role context
- Role: `vendor`

Step 2 — Create Vendor Profile

- Route: `/dashboard/profile`
- API: `PUT /api/vendor/profile`
- Fields: company name, slug, website, logo, tagline, description, founded year, HQ, funding stage, social links

Step 3 — Verification

- Vendor profile starts as `status: pending`
- Admin verifies → `verified: true` badge appears on all listings

4.2 Listing an App

Route: `/dashboard/products` → "Add Product"
API: `POST /api/vendor/apps`

Steps:

1. Click "Add Product" in vendor dashboard
2. Fill in listing form:
    - **Basic:** name, slug, tagline, short description, long description
    - **Category & Tags:** primary category, up to 10 tags, target audience
    - **Pricing:** type (free/freemium/paid/per-seat/custom), value, period, pricing tiers (JSON)
    - **Media:** logo URL, screenshots (array), demo URL, website URL
    - **Features:** key features list
    - **Integrations:** connected tools list
    - **Trust:** security certifications, compliance score, support email
3. Submit → listing enters `status: pending` in `app_listings`
4. Admin reviews and approves (see §5.2)
5. On approval → listing goes live at `/marketplace/app/[slug]`

AI-Assisted Listing:

- Route: `/list-product`
- API: `POST /api/ai-listing/submit` or `POST /api/listings/scrape`
- Paste your website URL → AI extracts: name, description, features, pricing
- Review and submit the pre-filled form

4.3 Managing Listings

Route: `/dashboard/products`
APIs:

- `GET /api/vendor/apps` — list all your apps
- `PUT /api/vendor/apps/[id]` — update listing
- `DELETE /api/vendor/apps/[id]` — delete listing

Steps:

1. View all products at `/dashboard/products`
2. Click any listing to edit: update description, pricing, screenshots, features
3. Changes go back to `pending` review (or auto-approve if minor)
4. Delete removes listing (soft-delete in DB)

4.4 Responding to Enquiries

Route: `/dashboard/enquiries`
APIs:

- `GET /api/vendor/enquiries` — list incoming enquiries

- `GET /api/enquiries/[id]/messages` — open thread
- `POST /api/enquiries/[id]/messages` — send reply

Steps:

1. Email notification arrives when buyer sends enquiry
2. Open `/dashboard/enquiries` → see all enquiries with status
3. Click enquiry → read buyer's message, context (company, job title)
4. Type reply → `POST /api/enquiries/[id]/messages`
5. Continue conversation until `closed`
6. Track enquiry-to-demo-to-deal conversion in analytics

4.5 Replying to Reviews

Route: `/dashboard/reviews`
APIs:

- `GET /api/apps/[id]/reviews` — fetch reviews for your app
- `POST /api/apps/[id]/reviews/[reviewId]/reply` — post vendor reply
- `POST /api/ai/review-reply` — get AI-suggested reply

Steps:

1. Navigate to `/dashboard/reviews`
2. See all reviews: star rating, content, verification status
3. Click "Reply" on any review
4. Optionally: click "Suggest Reply" → AI drafts a professional response (`POST /api/ai/review-reply`)
5. Edit and submit the reply
6. Reply appears publicly under the review

4.6 Buyer Intent Signals

Route: `/dashboard/analytics` → Intent Signals tab
API: `GET /api/vendor/intent-signals`

Signal types stored in `buyer_intent_events`:

Signal	Strength
App page view > 3 min	Warm
Pricing page click	Strong
Demo request	Strong
Competitor comparison view	Warm
Review read (5+ reviews)	Cold
Stack add	Strong
Enquiry sent	Hot

Steps:

1. Open `/dashboard/analytics` → Intent Signals
2. View list of anonymous/identified buyers showing interest
3. See signal type, strength, timestamp, buyer company/role (if available)
4. Act on Hot/Strong signals: reach out directly or trigger outbound campaign

---

4.7 Vendor Analytics Dashboard

Route: `/dashboard/analytics`
API: `GET /api/vendor/analytics`, `GET /api/vendor/health-score`

Metrics available:

- App page views (daily/weekly/monthly)
- Review count and average rating trend
- Enquiry volume and conversion rate
- Intent signal breakdown
- Top buyer segments (company size, role, industry)
- Health score: composite of rating, response time, listing completeness

---

4.8 Team Management

Route: `/dashboard/team`
APIs:

- `GET /api/vendor/team` — list team members
- `POST /api/vendor/team/invite` — invite member
- `DELETE /api/vendor/team/[id]` — remove member

Steps:

1. Go to `/dashboard/team`
2. Click "Invite Member" → enter email and role (owner, editor, viewer)
3. Invite email sent → member joins and sees vendor dashboard
4. Remove members at any time

4.9 Responding to RFPs

Route: `/rfp/[slug]` → "Submit Proposal" button
API: `POST /api/rfp/[slug]/respond`

Steps:

1. Browse RFPs at `/rfp` filtered by your category
2. Open an RFP → review requirements, budget, deadline
3. Click "Submit Proposal" → enter proposal details, pricing, timeline
4. Response stored in `rfp_responses` table
5. Buyer notified; vendor tracks status in `/dashboard/enquiries`

4.10 Publishing News & Updates

Route: `/dashboard` → News section
APIs:

- `POST /api/news` — create post
- `GET /api/vendor/news` — list your news posts
- `PUT /api/news/[id]` — update post

Post types: `feature_release`, `update`, `announcement`, `press`

Steps:

1. Click "New Post" in vendor news section
2. Fill in: title, excerpt, body (markdown), cover image, post type, tags
3. Submit → enters `status: pending` for admin approval
4. Admin approves → post live at `/news/[slug]`
5. Community can react (upvote) via `POST /api/news/[id]/react`

4.11 Upload Customer List (Verified Reviews)

Route: `/dashboard/settings` or during listing setup
API: `POST /api/vendor/customers/upload`

Steps:

1. Export customer emails from your CRM
2. Upload CSV → server hashes all emails (SHA-256) and stores in `vendor_customer_lists`
3. When a buyer submits a review, their email is hashed and matched
4. Match found → `purchase_verified = true` on the review → "Verified Purchase" badge

---

4.12 Win-Loss Analysis

Route: `/dashboard/analytics`
API: `POST /api/vendor/win-loss`

Steps:

1. After a deal closes (won or lost), submit outcome
2. Track which competitors you lost to most
3. AI surfaces patterns in win/loss data

4.13 Market Intelligence

Route: `/dashboard` → Market Intelligence
API: `GET /api/vendor/intelligence`

Provides:

- Competitor listings and ratings
- Market category trends
- Buyer demand signals by segment
- Review sentiment compared to competitors
- Similar vendors at `/dashboard/similar-vendors`

4.14 Billing (Vendor Plan)

Route: `/dashboard/billing`
APIs:

- `POST /api/billing/checkout` — start Stripe checkout
- `GET /api/billing/subscription` — current plan
- `POST /api/billing/portal` — open Stripe billing portal

Plans: Free → Developer → Business (unlocks intent signals, analytics, team seats, priority placement)

---

5. Admin Workflow

5.1 Access

- Role: `admin` — granted via `PUT /api/admin/users/[id]/role`
- Dashboard shows admin-specific navigation items
- All `/api/admin/*` endpoints require `requireAdmin()` middleware

5.2 Approving App Listings

Route: `/dashboard/pending-apps`
APIs:

- `GET /api/admin/listings` — queue of pending apps
- `PUT /api/admin/listings/[id]/status` — approve or reject

Steps:

1. Open `/dashboard/pending-apps` — see all apps with `status: pending`
2. Click listing → review all fields: name, description, screenshots, category, pricing
3. Check for policy violations: spam, duplicate, misleading claims
4. **Approve:** `status → published` → app goes live in marketplace
5. **Reject:** provide reason → vendor notified by email
6. **Request changes:** leave admin note → vendor edits and resubmits

---

5.3 Review Moderation

Route: `/dashboard/reviews` (admin view)
APIs:

- `GET /api/admin/reviews` — all reviews pending approval
- `PATCH /api/admin/reviews/[id]` — edit review content
- `PUT /api/admin/reviews/[id]/status` — approve, reject, or hide

Steps:

1. Open admin review queue
2. See review with: content, authenticity score, flag count, reviewer info
3. Check for fake reviews, spam, inappropriate content
4. Approve clean reviews → go live
5. Reject violating reviews → removed with reason
6. Edit minor issues (e.g., profanity) without rejection

5.4 User Management

Route: `/dashboard/users`
APIs:

- `GET /api/admin/users` — list all platform users
- `PUT /api/admin/users/[id]/role` — change role (buyer/vendor/admin)

Steps:

1. Search/filter users by email, role, plan, join date
2. View user profile: listings, reviews, enquiries, subscription
3. Promote buyer to vendor, or grant admin access
4. Suspend account (status change in `users` table)

5.5 Auto-Discovery Queue

Route: `/dashboard` → Discovery Queue
APIs:

- `GET /api/admin/discovery/queue` — pending discovered apps
- `POST /api/admin/discovery/trigger` — trigger a new discovery run
- `POST /api/admin/discovery/[id]/approve` — approve discovered app
- `POST /api/admin/discovery/[id]/reject` — reject with reason

How discovery works:

1. Crawler scans GitHub releases, Product Hunt, and other sources
2. AI extractor (`ai-extractor.ts`) pulls: name, description, website, logo
3. Extracted data stored in `discovery_queue` with `confidence_score`
4. Admin reviews queue → approve creates a draft listing
5. Claim email sent to vendor (`claim_email_sent` flag) via `vendor_claim_tokens`
6. Vendor claims listing → becomes their managed product

5.6 News & Content Moderation

Route: Admin news queue
APIs:

- `GET /api/admin/news` — pending vendor news posts
- `PUT /api/admin/news/[id]/status` — approve, reject, or feature

Steps:

1. Review vendor-submitted news posts
2. Check for accuracy, policy compliance, promotional overreach
3. Approve → post goes live at `/news/[slug]`
4. Feature selected posts on homepage news section

5.7 Badges & Compliance

APIs:

- `POST /api/admin/badges/assign` — assign badge to app (e.g., "Editor's Choice", "Top Rated 2025")
- `POST /api/admin/compliance/[appId]` — set compliance score

Steps:

1. Assign editorial badges to outstanding apps

2. Set compliance scores after security audit (SOC 2, GDPR, HIPAA)
3. Scores displayed as trust signals on app detail pages

5.8 Platform Statistics

Route: `/dashboard/overview` (admin view)
API: `GET /api/admin/stats`

Metrics:

- Total users, buyers, vendors
- Active listings
- Review volume and average rating
- MRR (Monthly Recurring Revenue) from subscriptions
- Subscription breakdown by plan
- Intent event volume (last 30 days)
- Top performing apps

5.9 Platform Settings

Route: Admin settings panel
APIs:

- `GET /api/admin/settings` — read settings from `admin_settings` table
- `PUT /api/admin/settings` — update settings

Configurable settings:

- Feature flags (enable/disable AI features, discovery, RFPs)
- Email digest schedule
- Rate limit overrides
- Maintenance mode

5.10 Digest Newsletter

API: `POST /api/admin/digest/send`

Steps:

1. Admin triggers digest send from settings panel
2. System selects top apps, reviews, news since last digest
3. Personalized digest email sent to opted-in buyers
4. Buyers see personalized feed at `/dashboard/digest`

5.11 Demo Bookings Management

Route: Admin demos panel
API: `GET /api/admin/demos`

Steps:

1. View all demo booking requests submitted via `POST /api/demos`
2. Assign to sales team
3. Mark as scheduled, completed, no-show

---

6. Authentication Flows

6.1 Email/Password Login

```
/login → POST /api/auth/login
  → validate credentials (scrypt hash compare)
  → check account lockout (>10 failures in 30 min → 15-min lock)
  → create session record (sessions table, 14-day TTL)
  → set HttpOnly session cookie
  → redirect to /dashboard
```

6.2 OAuth Login (GitHub / Google)

```
Click "Sign in with GitHub" → GET /api/auth/oauth/github
  → redirect to GitHub authorization page
  → GitHub redirects to GET /api/auth/oauth/github/callback
  → exchange code for access token
  → fetch user profile from GitHub API
  → upsert user in users table (email match → link, no match → create)
  → create session → redirect to /dashboard
```

6.3 Registration

```
POST /api/auth/register
  → validate: email unique, password strength
  → hash password (scrypt)
  → insert users row (role: buyer)
  → send verification email (email_verifications token)
  → create session → set cookie → redirect to /dashboard
```

6.4 Password Reset

```
POST /api/auth/forgot-password (email)
  → generate reset token (SHA-256 hashed in DB)
  → send email with link: /reset-password?token=…

GET /reset-password?token=…
  → verify token not expired, not used
  → show reset form

POST /api/auth/reset-password (token, newPassword)
  → hash new password
  → update users.password_hash
  → mark token used
  → invalidate all existing sessions for user
```

6.5 Session Management

- Sessions stored in `sessions` table

- `requireUser()` server utility reads session cookie on every protected API call
- Session cleanup background task: `session-cleanup.ts` purges expired rows
- Optional Redis: `redis.ts` for distributed session storage

---

7. AI Features

7.1 AI Copilot

Route: `/dashboard/copilot`
API: `POST /api/ai/copilot`
Composable: `useAICopilot()`
Rate limit: 60 requests/hour

Workflow:

1. User types natural language requirement
2. System builds prompt with: user profile, existing stack, budget, category preferences
3. Claude returns: 3–5 matched apps with reasoning, comparison table, recommendation
4. Session persisted in `ai_match_sessions` (for continuity and lead scoring)

7.2 Review Synthesis

API: `POST /api/ai/review-synthesis`
Cache: `review_synthesis_cache` table (expires after set interval)

On app detail page, AI summarizes all reviews into:

- Consensus opinion
- Power-user perspective
- Deal-breakers
- Best for / Worst for segments
- Sentiment trend (improving/declining)

7.3 AI Review Reply

API: `POST /api/ai/review-reply`

Vendor receives a review → clicks "Suggest Reply" → AI drafts a professional, empathetic response based on review content and sentiment.

7.4 Negotiation Tips

API: `POST /api/ai/negotiation`
Table: `negotiation_briefs`

AI analyzes: app pricing tiers, review data, market benchmarks → generates:

- Opening offer suggestion
- Leverage points (competitor pricing, low ratings, usage caps)
- Walk-away price recommendation
- Negotiation script

7.5 App Summary / Listing Copy

API: `POST /api/ai/app-summary`

Vendor inputs raw notes → AI generates:

- SEO-optimized short description
- Long description with feature highlights
- Key features list
- Tag suggestions

7.6 Buying Brief

API: `POST /api/ai/briefing`

Buyer describes needs → AI generates an executive PDF-ready buying brief:

- Requirements summary
- Shortlisted apps with pros/cons
- Recommended choice with justification
- Budget and timeline

7.7 AI-Powered Auto-Discovery

API: `POST /api/admin/discovery/trigger`
Util: `ai-extractor.ts`

Scans external sources (GitHub, Product Hunt) → AI extracts app data → populates `discovery_queue` for admin review.

7.8 AI Thumbnail Generation

APIs:

- `POST /api/ai/dalle-thumbnail` — DALL-E 3
- `POST /api/ai/stability-thumbnail` — Stability AI

Generates app thumbnails/hero images from text description.

---

8. Billing & Subscription Flow

8.1 Checkout

```
POST /api/billing/checkout (plan: 'developer' | 'business')
  → create Stripe Checkout Session
  → redirect buyer/vendor to Stripe hosted page
  → user enters card details on Stripe
  → success → Stripe fires webhook
```

8.2 Webhook Processing

```
POST /api/billing/webhook (Stripe event)
  → verify Stripe signature
  → handle events:
    checkout.session.completed → create user_subscriptions row
    invoice.paid → update current_period_end
```

```
    customer.subscription.updated → update plan/status
    customer.subscription.deleted → mark canceled
```

8.3 Subscription Management

```
POST /api/billing/portal
  → create Stripe Customer Portal session
  → redirect user to Stripe portal
  → user can: upgrade/downgrade plan, update payment method, cancel subscription
```

8.4 Refund Request

```
POST /api/billing/refund
  → validate within refund window
  → create refund via Stripe API
  → update user_subscriptions.status
  → send confirmation email
```

---

9. Trust & Verification System

9.1 Review Authenticity Scoring

Util: `trustEngine.ts` → `computeReviewAuthenticity()`

Factors:

Factor	Weight
Verified purchase (email match)	+40 pts
Complete user profile	+20 pts
Reviewer has multiple reviews	+10 pts
Review length > 100 words	+10 pts
Detailed pros/cons provided	+10 pts
Account age > 30 days	+10 pts

Score 0–100 displayed as trust indicator on each review.

9.2 Verified Purchase Flow

1. Vendor uploads customer CSV → server hashes all emails → stored in `vendor_customer_lists`
2. Buyer submits review → system hashes buyer email
3. Hash lookup in `vendor_customer_lists` for that `app_id`
4. Match → `purchase_verified = true` → "Verified Customer" badge on review

9.3 Vendor Claim Flow (Auto-Discovered Apps)

1. Admin approves discovered app from `discovery_queue`
2. System sends claim email to vendor contact at `claim_email_sent`

3. Vendor receives email with unique claim link (`vendor_claim_tokens`)
4. Vendor clicks link → token verified → listing transferred to their account
5. Vendor can then edit and manage the listing

9.4 Security Certifications

Stored in `app_listings.security_certs` (JSON array).
Supported: SOC 2, ISO 27001, HIPAA, GDPR, PCI-DSS.
Displayed as trust badges on app detail page.

---

10. Developer / API Access

10.1 API Key Creation

Route: `/developer`
APIs:

- `POST /api/v1/keys` — create API key
- `GET /api/v1/keys` — list API keys

Tiers:

Tier	Daily Limit	Features
free	100 requests	Public app data
developer	1,000 requests	Reviews, categories, agents
business	10,000 requests	All data + webhooks

10.2 V1 Public API Endpoints

```
GET /api/v1/apps          — list all apps (paginated)
GET /api/v1/apps/[slug]   — get app by slug
GET /api/v1/categories    — list categories
GET /api/v1/agents/discover — discover AI agents
```

Authentication: `Authorization: Bearer <api-key>` header.
Key validated by `apiKeyAuth.ts` → checks hash in `api_keys` table → checks daily quota → increments `requests_today` .

10.3 Embed Widget

File: `public/embed.js`
Route: `/embed/app/[id]`

Vendors can embed app showcase widgets on external websites:

```
<script src="https://moonmart.ai/embed.js" data-app-id="[id]"></script>
```

Widget renders a minimal app card with rating, pricing, and CTA.

10.4 LLM-Friendly Export

Route: `/api/llms-full.txt`

Full plain-text export of all published app listings, optimized for LLM ingestion and AI agent discovery.

Quick Reference: Key Database Tables

Table	Purpose
`users`	All user accounts with role and plan
`vendor_profiles`	Vendor company profiles
`app_listings`	All SaaS app listings
`reviews`	User reviews with trust scores
`sessions`	Active login sessions
`user_favorites`	Buyer saved apps
`user_stacks`	Buyer software stacks
`enquiries` + `enquiry_messages`	Buyer↔vendor messaging
`buying_rooms`	Team evaluation rooms
`contracts`	Buyer SaaS contracts
`rfps` + `rfp_responses`	Request for proposals
`questions` + `question_answers`	Q&A forum
`ai_match_sessions`	AI copilot history
`buyer_intent_events`	Buyer signal tracking
`discovery_queue`	Auto-discovered apps
`user_subscriptions`	Stripe subscription state
`api_keys`	Developer API keys
`news_posts`	Vendor news & updates
`events`	Platform events
`admin_settings`	Platform configuration

Last updated: 2026-05-02 | Platform: moonmart.ai